Gestor de gastos: el objeto gasto

Introducción a la práctica

Los casos prácticos extendidos de las primeras unidades van a dedicarse a la creación de una aplicación de manera **iterativa**. Dicha aplicación consiste en un **gestor de presupuesto personal**, en el que el usuario podrá ir anotando sus gastos, de manera que pueda consultarlos y llevar un seguimiento de ellos. Para eso utilizaremos un **único repositorio**, sobre el que iremos añadiendo nuevas funcionalidades en cada práctica. De esta forma, podremos ir estudiando los fundamentos del lenguaje JavaScript y las herramientas de desarrollo de cliente *(front-end)* en el contexto de una aplicación real, donde la mayoría de código que vayamos creando tendrá una función y una utilidad específica dentro de dicha aplicación.

EJEMPLO

En la documentación asociada al Estudio del caso, dispones del archivo **`GESTOR_PRESUPUESTO.zip`**, con el código base del repositorio.

El proceso que llevaremos a cabo está enmarcado dentro del **contexto educativo**, por lo que no debe confundirse con el proceso real de creación de aplicaciones de carácter profesional. En ocasiones, se propondrá la creación de varias versiones del mismo código utilizando técnicas distintas, o se tendrán que llevar a cabo rediseños o mejoras para ilustrar nuevos conceptos conforme los vayamos estudiando.

A grandes rasgos, los pasos que seguiremos para la elaboración de la aplicación serán los siguientes:

1. Creación del código y las funciones necesarias para el funcionamiento de la lógica de la aplicación. Se implementará como **librería**, por lo que se limitará a exportar una serie de objetos y funciones exclusivamente.
2. Creación de una **interfaz** web para mostrar la información de la aplicación en el navegador. En este punto podremos cargar nuestro código en el **navegador web** y ver los resultados en HTML.
3. Añadir los **controles de formulario** necesarios para **interactuar** con la aplicación.
4. Añadir mecanismos de **validación** para controlar la correcta introducción de datos por parte del usuario.
5. **Almacenar** los datos de la aplicación en el **navegador web**. Desde este momento, el usuario podrá guardar su información en el navegador: al recargar la página aparecerá la información que hubiera introducido (hasta entonces, cada vez que se recargaba la página se borraban todos los datos).
6. Conectarse con una **API web** para almacenar la información del usuario en un **servidor web externo**, incluyendo autenticación de usuario. De esta manera, el usuario podrá acceder a sus datos desde cualquier navegador.
7. Utilizar **librerías externas** para mostrar determinada información de manera **gráfica**, en lugar de texto exclusivamente.

Los requisitos de software de las prácticas son:

- Navegador **Chrome**.
- Entorno de ejecución **NodeJS**.

IMPORTANTE

Aparte de ejecutar los test, también puedes (y debes) probar las funciones que vayas realizando. Para ello puedes hacer uso del REPL de NodeJS o el navegador, además de utilizar páginas HTML que carguen la librería de la aplicación para poder acceder a sus funciones.

Se seguirán los principios del **desarrollo guiado por test**. Para ello se proporciona en el repositorio una serie de ficheros que permiten **ejecutar test**. Dichos test comprobarán que el programa cumple con algunos de los requisitos de la práctica.

Utilizaremos **NodeJS** para lanzar los test. Por ello, la primera vez que se acceda al código para trabajar habrá que **instalar las dependencias** mediante `npm install`. Para esta primera práctica, los test se ejecutan con `npm run test1`. Si se está utilizando **Visual Studio Code**, se pueden **ejecutar los test desde el propio editor** y utilizar ***breakpoints*** para interrumpir la ejecución y **depurar** el programa. Las configuraciones de lanzamiento creadas para ello están en el archivo `.vscode/launch.json` del repositorio.

IMPORTANTE

Es muy importante que utilices el **control de versiones** para ir guardando los cambios que vayas haciendo. Para ello, **crea un repositorio en la carpeta del proyecto** mediante `git init` y **configura un remoto** para sincronizar los cambios con un servicio como GitHub.

Tareas para realizar

Crea un fichero llamado `gestionPresupuesto.js` en la carpeta `js` del repositorio. A no ser que se indique lo contrario, todo el código que se cree deberá guardarse en este fichero.

Realiza las siguientes tareas:

- Crea una variable global en el fichero, con el nombre `presupuesto` y valor inicial `0`.
- Crea las siguientes **funciones**:
  - `actualizarPresupuesto`. Función de **un parámetro**, que se encargará de actualizar la variable global `presupuesto`. Esta función **comprobará** que el valor introducido es un **número no negativo**: en caso de que sea un dato válido, actualizará la variable `presupuesto` y **devolverá el valor de este**; en caso contrario, **mostrará un error por pantalla** y **devolverá el valor** −1.
  - `mostrarPresupuesto`. Función **sin parámetros**, que se encargará de devolver el texto siguiente: `Tu presupuesto actual es de X €`, en el que `X` es el valor de la variable global `presupuesto`.
  - `CrearGasto`. **Función constructora**, que se encargará de crear un objeto **gasto**. Esta función devolverá un objeto de tipo **gasto**. Deberá **comprobar que el valor introducido sea un número no negativo**; en caso contrario, asignará a la propiedad `valor` el valor 0.
- Objeto **gasto**. El objeto **gasto** creado por la función constructora `CrearGasto` tendrá las siguientes propiedades:
  - `descripcion`. Almacenará la descripción del gasto en formato **cadena**.
  - `valor`. Almacenará el valor del gasto en formato **numérico**.
  - `mostrarGasto`. Función sin parámetros, que muestre el texto `Gasto correspondiente a DESCRIPCION con valor VALOR €`, en el que `VALOR` y `DESCRIPCION` son las propiedades del objeto correspondientes.
  - `actualizarDescripcion`. Función de **un parámetro**, que actualizará la descripción del objeto.
  - `actualizarValor`. Función de **un parámetro**, que actualizará el valor del objeto. Se encargará de **comprobar que el valor introducido sea un número no negativo**; en caso contrario, dejará el valor como estaba.

Claves de resolución

Se proporciona el esqueleto del archivo `js/gestionPresupuesto.js`. En él se exportan las funciones que se tienen que crear para que se puedan ejecutar los test. El concepto de exportación lo estudiaremos más adelante.

Lo primero que tienes que hacer es descomprimir el fichero `zip` dentro de tu carpeta de usuario. Lo ideal es que no utilices rutas con espacios en blanco, porque en ocasiones crean problemas.

A continuación, accede a la ruta de la carpeta del repositorio y ejecuta **`npm install`** para instalar las dependencias. Ejecuta a continuación los test mediante **`npm run test1`** y comprueba que se lanzan pero que no se superan (como es lógico, ya que el código no está escrito).

Dedica un tiempo a familiarizarte con el código proporcionado. Observa la estructura de carpetas y el archivo del primer test.

Por último, empieza a escribir el código que se pide en el enunciado. Cada vez que creas haber completado una de las tareas que se piden, lanza los test mediante **`npm run test1`** y comprueba si superas alguno nuevo. Ve realizando *commits* en el repositorio de manera periódica; sobre todo, cuando consigas superar un nuevo test.